

The dataset used in this project is the NASA FIRMS (Fire Information for Resource Management System) VIIRS active fire data. It contains real-time satellite detections of wildfires, including latitude, longitude, brightness temperature, Fire Radiative Power (FRP), acquisition date/time, and confidence levels. For this project, approximately 8,007 fire detections from February 20 to March 5, 2026 across the Southern United States, Mexico, and Central America were analyzed.

I found this dataset interesting because wildfires are a major environmental and societal issue, especially growing up in California. Being able to analyze real satellite data and simulate how fires spread provides valuable insight into real-world disaster monitoring and predictions!

Hypothesis

Fires with higher initial brightness and Fire Radiative Power (FRP) will spread more aggressively than lower-intensity fires. This is rooted in the idea that brighter, more energetic fires release more heat and are therefore more likely to ignite surrounding vegetation and grow rapidly.

The dataset provides precise measurements of each fire's initial intensity (brightness and FRP) along with exact geographic locations. By feeding this real-world data into the automated fire spread simulation and performing statistical analysis, we can test whether higher initial intensity correlates with greater simulated spread and risk.

Data Analysis Techniques

Exploratory Data Analysis (EDA): Spatial mapping, histograms, and time-series aggregation will help us understand fire distribution, temporal patterns, and intensity ranges.

Threshold-based Classification & Binning: Fires were binned into High/Medium/Low intensity and High/Low risk bins.

Linear Regression: Used to model the relationship between brightness (independent variable) and FRP (dependent variable)

Simulation Modeling: A grid-based cellular model was used to simulate fire spread based on real fire detections, allowing us to observe real intensity vs simulated behavior.

1. Scatter Plot - Fire Locations (Lat vs Lon)

BLANK = 0 fires | . = 1 fire | * = ≤5 fires | # = >5 fires

```
## # *
# . . #      * . .
. * # . . # * . . #
# . * * # * * * * * * *
# . * * . * * * * * * *
# . * * * * * * * * * *
* . * * * * * * * * *
# . * * * * * * * * *
# * * * * * * * * *
* . * * * * * * * *
* . * * * * * * * *
. * * * * * * * *
# * * * * * * * *
# * * * * * * * *
# * * * * * * * *
```

This scatter plot visualizes the geographic distribution of all (~) 8 thousand fires detected across the region. It clearly shows that wildfires aren't randomly scattered but instead form distinct regional clusters, particularly in Northern Mexico, Southern Texas, Florida, and parts of the Southwest. The density map helps identify wildfire hotspots and demonstrates that certain geographic areas are significantly more prone to fire activity during this time period.

2. Histogram of Fire Brightness Values

```
290-297: (0)
298-305: ##### (1228)
306-313: ##### (2387)
314-321: ##### (1957)
322-329: ##### (1218)
330-337: ##### (476)
338-345: ### (290)
346-353: ## (164)
354-361: # (101)
362-369: (60)
```

The brightness histogram reveals the intensity distribution of the detected fires. The majority of fires fall in the moderate range of 306-321 brightness, while a smaller, important subset reaches very high intensities above 350. The distribution indicates that while most fires are relatively average in strength, there exists a notable number of intense fires that could pose a greater risk of rapid spread. Understanding this distribution is key to assessing overall wildfire danger in the dataset.

3. Time Series - Number of Fires per Day

```
2026-02-20: ##### (667 fires)
2026-02-21: ##### (358 fires)
2026-02-22: ##### (369 fires)
2026-02-23: ##### (702 fires)
2026-02-24: ##### (445 fires)
2026-02-25: ##### (540 fires)
2026-02-26: ##### (700 fires)
2026-02-27: ##### (920 fires)
2026-02-28: ##### (700 fires)
2026-03-01: ##### (464 fires)
2026-03-02: ##### (633 fires)
2026-03-03: ##### (501 fires)
2026-03-04: ##### (456 fires)
2026-03-05: ##### (552 fires)
```

This plot tracks the daily number of active fire detections from February 20 to March 5, 2026. It shows clear fluctuations in fire activity daily, with a noticeable peak around February 27 (920 fires). These temporal patterns suggest that fire occurrences are influenced by short-term environmental changes such as weather, wind, or dryness, highlighting the importance of continuous monitoring rather than static snapshots.

4. Initial Fire Intensity vs Spread Potential

```
High Intensity (Fast Spread): 836
Medium Intensity              : 4794
Low Intensity                  : 2377
```

Fires were categorized into three intensity levels based on brightness and FRP values. Out of ~8,000 fires, 836 were classified as high intensity (most likely to spread aggressively), 4,794 as medium intensity, and 2,377 as low intensity. This classification provides a practical risk assessment and shows that roughly 10% of detected fires have the potential to become major threats, which is valuable information for simulation and real-world response planning.

5. High-Risk vs Low-Risk Clustering

```
High-Risk Fires : 1612 (20.1324%)
Low-Risk Fires  : 6395
```

Using a threshold-based clustering technique, fires were divided into high-risk and low-risk groups. 1,600 fires (20.13%) met the criteria for high-risk (brightness ≥ 330 or FRP ≥ 50), while the remaining 6,395 were considered lower risk. This simple, effective clustering method helps prioritize which fires warrant closer in both the simulation and potential real-world scenarios.

6. Linear Regression - Brightness vs FRP

Regression Equation: $FRP = 2.9796 * Brightness + -917.4710$
Correlation Strength: Strong positive relationship

Visualization (Brightness -> Predicted FRP):
290 brightness → predicted FRP: -53.3745
310 brightness → predicted FRP: 6.2183
330 brightness → predicted FRP: 65.8112 #####
350 brightness → predicted FRP: 125.4041 #####
370 brightness → predicted FRP: 184.9969 #####
390 brightness → predicted FRP: 244.5898 #####

A linear regression model was applied to quantify the statistical relationship between fire brightness and Fire Radiative Power (FRP). The resulting equation:

$$FRP = 2.9796 * Brightness - 917.47$$

This equation shows a strong positive correlation, indicating that fires with higher initial brightness tend to release significantly more energy, supporting the idea that early satellite measurements can be used to predict fire behavior and potential spread.

Conclusion

The NASA FIRMS VIIRS wildfire dataset proved highly effective in helping us investigate our hypothesis. Through a combination of exploratory data analysis, statistical modeling, and fire spread simulation, we found clear evidence that fires with higher initial brightness and Fire Radiative Power (FRP) tend to be more dangerous and have greater spread potential.

The linear regression analysis revealed a strong positive relationship, confirming that initial satellite measurements can serve as reliable indicators of fire intensity. This was further supported by the simulation, where high-intensity fires consistently affected larger areas of the grid compared to weaker ones. The spatial clustering and temporal analysis also showed that wildfires are highly concentrated in certain regions and exhibit noticeable daily variation, showing the dynamic nature of fire activity.

Overall, the dataset successfully resolved my hypothesis. Higher brightness and FRP values are strong predictors of more aggressive fire behavior. This project demonstrates the values of integrating real satellite data with computational modeling for wildfire risk assessment.

Satellite remote sensing, when combined with data analysis and simulation techniques, provides a powerful framework for understanding and predicting wildfire spread. Future work could enhance this model by incorporating additional variables such as wind speed, vegetation type, and terrain to improve prediction accuracy.

Fire Point Header

```
#ifndef FIREPOINT_H
#define FIREPOINT_H
#include <string>
using namespace std;
struct FirePoint {
    double lat;           // Latitude
    double lon;           // Longitude
    double brightness;    // Brightness temperature
    double frp;           // Fire Radiative Power (FRP)
    string date;          // Acquisition date
};
#endif
```

Data Loader Header

```
#ifndef DATALOADER_H
#define DATALOADER_H
#include <vector>
#include <string>
#include "FirePoint.h"
using namespace std;
class DataLoader {
public:
    // Loads a single CSV file and returns all fire points from it
    vector<FirePoint> loadFile(const string& filename);
    // Loads multiple CSV files and combines all fire points into one vector
    vector<FirePoint> loadMultipleFiles(const vector<string>& files);

private:
    // Parses one line of the CSV into the FirePoint struct...helper function
    FirePoint parseLine(const string& line);
};
#endif
```

Data Loader CPP

```
#include "DataLoader.h"
#include <fstream>
#include <sstream>
#include <iostream>
using namespace std;
FirePoint DataLoader::parseLine(const string& line) {
    FirePoint f = {0.0, 0.0, 0.0, 0.0, ""};
    stringstream ss(line);
    string token;
    vector<string> tokens;

    // col order: 0 lat, 1 lon, 2 bright, 11 frp
    while (getline(ss, token, ',')) {
        tokens.push_back(token);
    }

    if (tokens.size() < 12) { return f; } // invalid line

    try {
```

```

        f.lat = stod(tokens[0]);
        f.lon = stod(tokens[1]);
        f.brightness = stod(tokens[2]);
        f.frp = stod(tokens[11]);
        f.date = tokens[5];
    } catch (...) {
        // skipping bad lines here
    }

    return f;
}

```

The `parseLine` function splits each CSV line by commas into a vector `tokens`. It then extracts (safely) the following fields: latitude [0], longitude [1], brightness [2], frp [11], and acquisition date [5]. Try-catch is implemented to skip any malformed lines. The function returns a `FirePoint` struct (0 if parsing fails).

```

vector<FirePoint> DataLoader::loadFile(const string& filename) {
    vector<FirePoint> data;
    ifstream ifile(filename);

    if (!ifile.is_open()) {
        cout << "Error opening file: " << filename << endl << "Please try again!" << endl;
        return data;
    }
    string line;
    getline(ifile, line); // skipping header

    int count = 0;
    while (getline(ifile, line)) {
        if (!line.empty()) {
            FirePoint fp = parseLine(line);
            // adding if coords valid
            if (fp.lat != 0.0 || fp.lon != 0.0) {
                data.push_back(fp);
                count++;
            }
        }
    }
    cout << "Parsed " << count << " valid fires." << endl;
    return data;
}

```

The `loadFile` function opens a CSV file, skips the header row, and reads every remaining line and passes it to `ParseLine()`. It only keeps valid fire records (with non-zero coordinates) and prints out a count of the successfully parsed fires for debugging.

```

vector<FirePoint> DataLoader::loadMultipleFiles(const vector<string>& files) {
    vector<FirePoint> allData;

    for (const string& file : files) {
        vector<FirePoint> temp = loadFile(file);
        allData.insert(allData.end(), temp.begin(), temp.end());
        cout << "Loaded " << temp.size() << " fires from " << file << endl;
    }
    cout << "Total fires loaded: " << allData.size() << endl;
}

```

```

    return allData;
}

```

The loadMultipleFiles function loops through all 14 inputs, combines all fire points from every file into one large vector using insert(), and prints the progress (fires per file plus grand total).

Grid Header

```

#ifndef GRID_H
#define GRID_H
#include <vector>
#include "FirePoint.h"
using namespace std;
enum CellType {
    EMPTY,      // Normal land
    FIRE,       // Currently burning
    BURNED,     // Previously burned
    WATER,      // Blocks fire
    ROAD,       // Road...Slows fire spread
    BUILDING    // Building...Increases fire spread
};
struct Cell {
    CellType type;
    double intensity;    // Strength of the fire
    bool isNew = false; // Used to distinguish new fires for visualization
};
class Grid {
private:
    static const int WIDTH = 150;    // Grid width...can adjust
    static const int HEIGHT = 150;   // Grid height...can adjust

    Cell cells[HEIGHT][WIDTH];       // 2D array
    // Helper functions to convert real-world coordinates to grid indices
    int mapX(double lon);
    int mapY(double lat);

public:
    Grid(); // Constructor

    void seedFire(const vector<FirePoint>& fires); // Places real fires on grid
    void spreadFire(); // Simulates one step of fire spread
    void display(); // Prints the current state of the grid

    int countBurning() const; // Returns number of cells currently on fire
    int countBurned() const; // Returns number of burned cells
};
#endif

```

CellType enumerates 6 different types of terrain that affects how fire behaves. The Cell struct represents one cell on the map, stores its type, fire intensity, and whether it's a new fire (for coloring in display()). Cells[height][width] is a fixed 2D array that acts as the simulation world. MapX()/mapY() converts the real latitude/longitude from satellite data into grid positions. SeedFire() takes the loaded FirePoint data and places initial fires on the grid. SpreadFire() spreads fire to neighboring cells based on intensity and terrain rules. Display() visualizes the current state of simulation with a red F (new fire) or a yellow F (ongoing fires).

Grid CPP

```
#include "Grid.h"
#include <iostream>
#include <iomanip>
#include <cstdlib>
#include <ctime>
#define RED      "\033[31m"
#define YELLOW   "\033[33m"
// Constructor...initializes random terrain
Grid::Grid() {
    srand(static_cast<unsigned int>(time(nullptr))); // random seed
    for (int y = 0; y < HEIGHT; y++) {
        for (int x = 0; x < WIDTH; x++) {
            int r = rand() % 100;

            if (r < 8) {
                cells[y][x].type = WATER;           // ~8% water (fire blocker)
            } else if (r < 18) {
                cells[y][x].type = ROAD;             // ~10% roads (slows fire)
            } else if (r < 25) {
                cells[y][x].type = BUILDING;         // ~7% buildings (burns faster)
            } else {
                cells[y][x].type = EMPTY;           // ~75% normal land
            }
            cells[y][x].intensity = 0;
        }
    }
}

int Grid::mapX(double lon) {
    return static_cast<int>((lon + 115.0)) * (WIDTH / 50.0);
}

int Grid::mapY(double lat) {
    return static_cast<int>((46.0 - lat) * (HEIGHT / 29.0));
}
```

MapX and mapY convert the real-world latitude and longitude from satellite data into grid positions.

```
void Grid::seedFire(const vector<FirePoint>& fires) {
    int placed = 0;
    for (const auto& f : fires) {
        int x = mapX(f.lon);
        int y = mapY(f.lat);

        if (x >= 0 && x < WIDTH && y >= 0 && y < HEIGHT) {
            cells[y][x].type = FIRE;
            cells[y][x].intensity = f.brightness;
            cells[y][x].isNew = true;
            placed++;
        }
    }
    cout << "Successfully placed " << placed << " fires on the grid!" << endl;
}
```

The seedFire() function loops through every fire point in the input list, converts their real

coordinates into grid positions, checks if the mapped positions are valid within the grid, and marks the grid cell with a ‘new’ fire, as well as storing how strong/bright the fire is. It then counts how many fires are added.

```
void Grid::spreadFire() {
    Cell newGrid[HEIGHT][WIDTH];

    for (int y = 0; y < HEIGHT; y++) {
        for (int x = 0; x < WIDTH; x++) {
            newGrid[y][x] = cells[y][x]; // copy everything
            newGrid[y][x].isNew = false; // reset new flag
        }
    }

    for (int y = 1; y < HEIGHT - 1; y++) {
        for (int x = 1; x < WIDTH - 1; x++) {
            if (cells[y][x].type == FIRE && !cells[y][x].isNew) {
                double intensity = cells[y][x].intensity;

                auto trySpread = [&](int ny, int nx) {
                    if (cells[ny][nx].type == WATER ||
                        cells[ny][nx].type == FIRE ||
                        cells[ny][nx].type == BURNED) { return; }
                    double chance = intensity;

                    if (cells[ny][nx].type == ROAD) { chance *= 0.25; } // 25% of normal chance
                    if (cells[ny][nx].type == BUILDING) { chance *= 1.6; } // 160% of normal chance

                    double prob = chance / 800.0; // anything 800 brightness & above means 100% spread
                    chance

                    if (prob > 1) { prob = 1; }

                    if ((rand() / (double)RAND_MAX) < prob ) {
                        newGrid[ny][nx].type = FIRE;
                        newGrid[ny][nx].intensity = intensity;
                        newGrid[ny][nx].isNew = true;
                    }
                };
                trySpread(y+1, x);
                trySpread(y-1, x);
                trySpread(y, x+1);
                trySpread(y, x-1);

                // only burn out sometimes, not always!
                double burnOutChance = 0.35; // the higher this is the faster it will burn out
                if ((rand() / (double)RAND_MAX) < burnOutChance) {
                    newGrid[y][x].type = BURNED;
                    // newGrid[y][x].isNew = false;
                }
            }
        }
    }
    // copying back
    for (int y = 0; y < HEIGHT; y++) {
        for (int x = 0; x < WIDTH; x++) {
            cells[y][x] = newGrid[y][x];
        }
    }
}
```

```

    }
}

```

The `spreadFire()` function simulates one step of fire propagation by first creating a temporary copy of the grid, then checks every burning cell and attempts to spread the fire to its four neighbors. The spread probability is based on the fire's intensity and the neighbor's terrain type (water blocks the most, roads reduce spread chance, and buildings increase it). Each burning cell also has an ~30% chance to burn out and become burned. Finally, the temporary grid is copied back to the grid.

```

void Grid::display() {
    #ifdef _WIN32
        system("color");
    #endif
    for (int y = 0; y < HEIGHT; y++) {
        cout << "|";
        for (int x = 0; x < WIDTH; x++) {
            const Cell& c = cells[y][x];
            char ch = '.';
            switch (c.type) {
                case FIRE: ch = 'F'; break;
                case BURNED: ch = '#'; break;
                case WATER: ch = '~'; break;
                case ROAD: ch = '='; break;
                case BUILDING: ch = 'B'; break;
                default: ch = '.'; break;
            }
            if (c.type == FIRE) {
                if (c.isNew) {
                    cout << "\033[31mF\033[0m"; // Red F = new fire
                } else {
                    cout << "\033[33mF\033[0m"; // existing fires = yellow F
                }
            } else {
                cout << ch; // normal for everything else
            }
        }
        cout << "|" << endl;
    }
    cout << "Burning: " << countBurning() << " | Burned: " << countBurned() << endl;
}

```

The `display()` function prints the current state of the 150x150 simulation grid to the console, it then loops through every cell in the 2D array and uses a switch statement to assign a display character: F (Fire), # (Burned), ~ (Water), = (Road), B (Building), and . (Empty). For fire cells, it checks the `isNew` flag and prints a red F for newly ignited fires or a yellow F for ongoing fires using ANSI color codes with all other terrain types printed in normal text. At the bottom, it prints the total number of burning cells and burned cells using the `countBurning()` and `countBurned()` functions.

```

int Grid::countBurning() const {
    int count = 0;
    for (int y = 0; y < HEIGHT; y++) {
        for (int x = 0; x < WIDTH; x++) {
            if (cells[y][x].type == FIRE) {
                count++;
            }
        }
    }
    return count;
}

int Grid::countBurned() const {
    int count = 0;
    for (int y = 0; y < HEIGHT; y++) {
        for (int x = 0; x < WIDTH; x++) {
            if (cells[y][x].type == BURNED) {
                count++;
            }
        }
    }
    return count;
}

```

The countBurning() function iterates through the 150x150 grid and counts how many cells currently have the type FIRE, returns the total as an int. The countBurned() function iterates through the 150x150 grid and counts how many cells have the type BURNED, returns the total as an int. These functions are called at the end of every simulation step in display().

Simulation Header

```

#ifndef SIMULATION_H
#define SIMULATION_H
#include <vector>
#include <string>
#include "FirePoint.h"
#include "Grid.h"
using namespace std;
class Simulation {
private:
    vector<FirePoint> fires;
    Grid grid;

public:
    // Loads wildfire data from multiple CSV files using DataLoader class & seeds fires onto the grid
    void loadData(const vector<string>& files);
    // Runs the fire spread simulation for 10 steps & prints each step
    void run(int steps);
    // Returns a const reference to the loaded FirePoint objects so Analysis class can use the data
    const vector<FirePoint>& getFires() const { return fires; }
};
#endif

```

Simulation CPP

```

#include "Simulation.h"
#include "DataLoader.h"
#include <iostream>
void Simulation::loadData(const vector<string>& files) {
    DataLoader loader;
    // Loads all fires from the 14 CSV files & stores them
    fires = loader.loadMultipleFiles(files);
    // Places the real satellite-detected fires onto simulation grid
    grid.seedFire(fires);
}
void Simulation::run(int steps) {
    cout << endl << "=== Fire Simulation Grid ===" << endl;
    // Printing legend for viewer
    cout << "Legend: \033[31mF\033[0m = New Fire | \033[33mF\033[0m = Ongoing Fire | # = Burned | ~ =
Water | = = Road | B = Building\n" << endl;
    // Main simulation loop
    for (int i = 0; i < steps; i++) {
        cout << "==== Step " << i << " ==== " << endl << endl;

        grid.display();          // Shows current state of the grid
        grid.spreadFire();       // Simulates fire spread for each step
        cout << endl << endl;
    }
    // Final summary after simulation ends
    cout << "\n==== SIMULATION COMPLETE ==== \n";
    cout << "Total steps: " << steps << endl;
    cout << "Final burning cells: " << grid.countBurning() << endl;
    cout << "Total burned cells: " << grid.countBurned() << endl;
    cout << "Total fires originally loaded: " << fires.size() << endl;
}

```

Analysis Header

```

#ifndef ANALYSIS_H
#define ANALYSIS_H
#include "FirePoint.h"
#include <vector>
class Analysis {
public:
    // Runs all analysis & prints the 6 plots + results
    static void runAnalysis(const std::vector<FirePoint>& fires);
private:
    // Plot 1: Visualizes geographic distribution of fires
    static void plotFireLocations(const std::vector<FirePoint>& fires);

    // Plot 2: Shows distribution of fire brightness values
    static void plotBrightnessHistogram(const std::vector<FirePoint>& fires);

    // Plot 3: Shows how many fires occurred each day (time series)
    static void plotTimeSeries(const std::vector<FirePoint>& fires);

    // Plot 4: Categorizes fires by intensity and spread potential
    static void plotIntensityVsSpread(const std::vector<FirePoint>& fires);

    // Plot 5: Groups fires into high-risk vs low-risk with thresholds
    static void plotRiskClustering(const std::vector<FirePoint>& fires);

    // Plot 6: Linear regression (Brightness vs FRP)

```

```

        static void plotLinearRegression(const std::vector<FirePoint>& fires);
    };
#endif

```

Analysis CPP

```

#include "Analysis.h"
#include <iostream>
#include <iomanip>
#include <algorithm>
#include <map>
void Analysis::runAnalysis(const std::vector<FirePoint>& fires) {
    if (fires.empty()) {
        std::cout << "No fires to analyze!\n";
        return;
    }
    std::cout << "\n=== WILDFIRE DATA ANALYSIS & VISUALIZATION ===\n";
    std::cout << "Total fires detected: " << fires.size() << "\n\n";
    plotFireLocations(fires); // Plot 1 - Geographic distribution
    plotBrightnessHistogram(fires); // Plot 2 - Brightness distribution
    plotTimeSeries(fires); // Plot 3 - Fires per day
    plotIntensityVsSpread(fires); // Plot 4 - Intensity categories
    plotRiskClustering(fires); // Plot 5 - Risk clustering
    plotLinearRegression(fires); // Plot 6 - Linear regression
    std::cout << "=== Data Analysis Complete ===\n\n";
}

// Plot 1: Fire Locations (Text Scatter)
void Analysis::plotFireLocations(const std::vector<FirePoint>& fires) {
    cout << "1) Scatter Plot - Fire Locations (Lat vs Lon)\n";
    cout << "BLANK = 0 fires | . = 1 fire | * = ≤5 fires | # = >5 fires" << endl;
    const int rows = 15, cols = 30;
    int grid[15][30] = {0}; // small grid for counting fire density
    // Map each fire's real coordinates to a small grid cell
    for (const auto& f : fires) {
        int y = static_cast<int>((48.0 - f.lat) * (rows / 33.0));
        int x = static_cast<int>((f.lon + 120.0) * (cols / 65.0));
        if (y >= 0 && y < rows && x >= 0 && x < cols) {
            grid[y][x]++; // Counting how many fires are in the cell
        }
    }
    // Printing the density map row by row
    for (int y = 0; y < rows; y++) {
        for (int x = 0; x < cols; x++) {
            int count = grid[y][x];
            if (count == 0) { cout << " "; } // Empty area
            else if (count == 1) { cout << "."; } // 1 fire
            else if (count <= 5) { cout << "*"; } // 2-5 fires
            else { cout << "#"; } // 6+ fires
        }
        cout << "\n";
    }
    cout << "\n";
}

```

```

// Plot 2: Histogram of Fire Brightness Values
void Analysis::plotBrightnessHistogram(const std::vector<FirePoint>& fires) {
    std::cout << "2) Histogram of Fire Brightness Values\n";
    int bins[10] = {0}; // 10 bins to count frequency of brightness values
    // Bin each fire's brightness (starting from 290, each bin covers 8 units)
    for (const auto& f : fires) {
        int bin = static_cast<int>((f.brightness - 290) / 8);
        if (bin >= 0 && bin < 10) bins[bin]++;
    }
    // Find the bin with the most fires (used to scale the bar lengths)
    int maxCount = 0;
    for (int i = 0; i < 10; i++) if (bins[i] > maxCount) maxCount = bins[i];
    // Print each bin as a bar chart
    for (int i = 0; i < 10; i++) {
        int start = 290 + i * 8;
        std::cout << std::setw(3) << start << "-" << (start + 7) << ": "
            << std::string(bins[i] * 30 / (maxCount + 1), '#')
            << " (" << bins[i] << ")\n";
    }
    std::cout << "\n";
}

// Plot 3: Time Series - Number of Fires per Day
void Analysis::plotTimeSeries(const std::vector<FirePoint>& fires) {
    std::cout << "3) Time Series - Number of Fires per Day\n";
    std::map<std::string, int> daily; // maps date string ... # of fires on that day
    // count how many fires occurred on each day
    for (const auto& f : fires) {
        daily[f.date]++; // grouping by date
    }

    // print each day with a bar representing the # of fires
    for (const auto& p : daily) {
        std::cout << " " << p.first << ": " // printing the date
            << std::string(p.second / 25 + 1, '#') // bar
            << " (" << p.second << " fires)\n";
    }
    std::cout << "\n";
}

// Plot 4: Initial Fire Intensity vs Spread Potential
void Analysis::plotIntensityVsSpread(const std::vector<FirePoint>& fires) {
    std::cout << "4) Initial Fire Intensity vs Spread Potential\n";
    int high = 0, med = 0, low = 0; // counters for intensity categories
    // classify each fire based on brightness and FRP
    for (const auto& f : fires) {
        if (f.brightness > 340 || f.frp > 80) high++; // high intensity
        else if (f.brightness > 310) med++; // medium intensity
        else low++; // low intensity
    }
    // Displaying the results
    std::cout << " High Intensity (Fast Spread): " << high << "\n";
    std::cout << " Medium Intensity : " << med << "\n";
    std::cout << " Low Intensity : " << low << "\n\n";
}

```

```

// Plot 5: High-Risk vs Low-Risk Fire Clustering
void Analysis::plotRiskClustering(const std::vector<FirePoint>& fires) {
    std::cout << "5) High-Risk vs Low-Risk Fire Clustering\n";
    int highRisk = 0; // counter for high-risk fires
    // threshold-based clustering
    for (const auto& f : fires) {
        // A fire is considered high-risk if it has high brightness or FRP
        if (f.brightness >= 330 || f.frp >= 50) highRisk++;
    }
    // display results with percentage
    std::cout << "    High-Risk Fires : " << highRisk
        << " (" << (highRisk * 100.0 / fires.size()) << "%)\n";
    std::cout << "    Low-Risk Fires : " << (fires.size() - highRisk) << "\n\n";
}

// 6. Linear Regression - Brightness vs FRP
void Analysis::plotLinearRegression(const std::vector<FirePoint>& fires) {
    std::cout << "6) Linear Regression - Brightness vs FRP\n";
    double sumX = 0, sumY = 0, sumXY = 0, sumX2 = 0;
    size_t n = fires.size(); // number of data points
    // calculating the sums needed for linear regression
    for (const auto& f : fires) {
        double x = f.brightness; // independent
        double y = f.frp;        // dependent
        sumX += x;
        sumY += y;
        sumXY += x * y;
        sumX2 += x * x;
    }
    // calculate slope (m) and intercept (b)
    double m = (n * sumXY - sumX * sumY) / (n * sumX2 - sumX * sumX); // slope
    double b = (sumY - m * sumX) / n; // intercept
    std::cout << std::fixed << std::setprecision(4);
    std::cout << "    Regression Equation: FRP = " << m << " * Brightness + " << b << "\n";
    // interpret strength of the relationship
    std::cout << "    Correlation Strength: ";
    if (std::abs(m) > 1.5) std::cout << "Strong positive relationship\n";
    else if (std::abs(m) > 0.8) std::cout << "Moderate positive relationship\n";
    else std::cout << "Weak relationship\n";
    // Visualization of regression line
    std::cout << "\n    Visualization (Brightness -> Predicted FRP):\n";
    for (int i = 290; i <= 400; i += 20) {
        double predicted = m * i + b;
        int barLength = static_cast<int>(predicted / 8.0);
        if (barLength < 0) barLength = 0; // prevents a negative bar length

        std::cout << "    " << std::setw(3) << i
            << " brightness -> predicted FRP: "
            << std::setw(7) << predicted
            << "    " << std::string(barLength, '#') << "\n";
    }
    std::cout << "\n";
}

```

Main CPP

```
#include <iostream>
#include "Simulation.h"
#include "Analysis.h"
int main() {
    Simulation sim;
    // List of all 14 data files containing NASA FIRMS wildlife detection data
    vector<string> files = {
        "51.txt", "52.txt", "53.txt", "54.txt", "55.txt", "56.txt", "57.txt", "58.txt", "59.txt",
        "60.txt", "61.txt", "62.txt", "63.txt", "64.txt"
    };

    // load all fire data from CSVs & seed them onto the grid
    sim.loadData(files);
    // run fire spread simulation for 10 steps
    sim.run(10);

    cout << "\nRunning data analysis and generating 6 plots...\n";
    // performing data analysis and generating the plots
    Analysis::runAnalysis(sim.getFires());

    cout << "\n==== PROJECT COMPLETE ==== \n";

    return 0;
}
```

Instructions for Downloading Data and Using Code

To download the dataset from NASA FIRMS use this link:

<https://firms.modaps.eosdis.nasa.gov/download/>

Then select your date range...choose 14 files.

Once your files have been downloaded, put them into the project directory with the code.

Then change the file name in main.cpp to your file names and run the code as is!